



AI Evaluation & Benchmarking

AICB-P1 · Ngày 14 · Đo lường chất lượng AI một cách khoa học

Tên Giảng Viên

VinUniversity · Phase 1 · 2026

HÃY SUY NGHĨ...

“Sếp hỏi: AI agent của mình tốt hơn ChatGPT bao nhiêu? Bạn nói sao nếu không có benchmark?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Evaluation fundamentals & eval-driven dev
2. Khi eval thất bại: bài học thực tế
3. Metrics cho AI agent
4. Retrieval eval: Hit Rate, MRR, NDCG
5. Benchmark design & golden datasets
6. Synthetic data generation (SDG)
7. Bức tranh benchmark công khai 2026
8. Agentic & trajectory evaluation
9. LLM-as-Judge & multi-judge consensus
10. RAGAS framework
11. Statistical rigor in eval
12. Eval economics & async
13. Regression release gates & CI/CD
14. Failure analysis & continuous improvement
15. Hands-on & key takeaways

- Hiểu **evaluation là engineering discipline** (evals = unit tests cho agent), không phải cảm tính
- Đánh giá **retrieval** bằng Hit Rate / MRR / NDCG trước khi đánh giá generation
- Thiết kế **benchmark** + sinh **golden dataset** bằng SDG có ground-truth doc IDs
- Đọc đúng **benchmark công khai 2026** (saturation, contamination, Goodhart) và biết vì sao phải tự xây eval
- Dùng **LLM-as-Judge + multi-judge consensus** (agreement rate, calibration) một cách khách quan
- Chạy **RAGAS**, kiểm định **thống kê** (CI, paired test), tối ưu **chi phí eval**
- Xây **regression release gate** tự động quyết định release / rollback

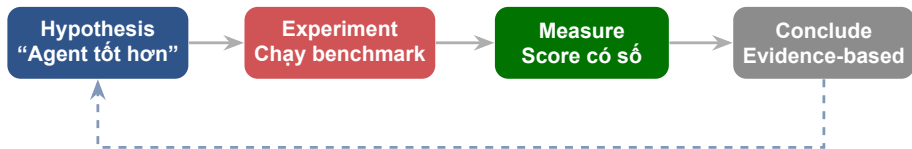
Một hệ thống đánh giá tự động cho agent: golden dataset, retrieval eval, multi-judge consensus, RAGAS scores, regression gate, và failure analysis — chứng minh bằng con số agent tốt ở đâu, yếu ở đâu.

- **Golden dataset** 50+ cases (SDG) kèm ground-truth doc IDs cho retrieval
- **Retrieval eval**: Hit Rate & MRR cho vector store
- **Multi-judge consensus**: ≥ 2 judge models, agreement rate, xử lý xung đột
- **Regression gate**: delta analysis + auto release/rollback (quality/cost/latency)
- **Failure analysis**: 5 Whys cho các worst cases + improvement log

Evaluation Fundamentals

Evaluation biến cảm nhận “agent trả lời tốt” thành số liệu lặp lại được, so sánh được, và gate được release.

01



Không đo = không cải thiện. Mỗi giả thuyết về chất lượng phải đi qua một experiment đo được, rồi kết luận bằng số — không phải bằng cảm giác.

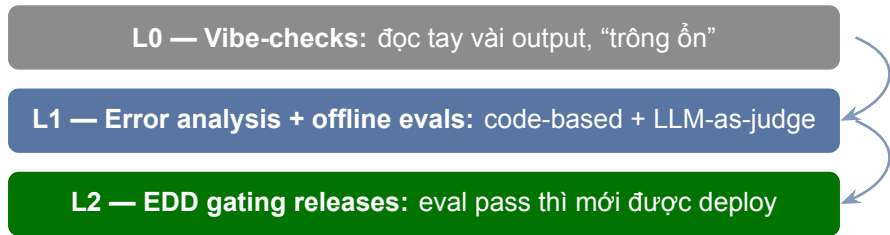
Vì sao không dùng unit test cũ?

- Agent là **probabilistic** + multi-step: cùng input, output khác nhau mỗi lần
- `assert output == expected` vỡ ngay — không có một chuỗi đúng duy nhất
- Ta chấm **outcome/behavior**, không chấm exact-match

Eval đóng vai trò của test suite: viết eval *trước*, để eval định nghĩa “đúng nghĩa là gì”, rồi mới optimize prompt/model.

Garry Tan (YC): “Evals are emerging as the real moat for AI startups”.

“Grade what the agent *produced*, not the *path* it took.” Ép đúng thứ tự bước là quá brittle — agent thường tìm cách hợp lệ mà người viết eval không lường trước.



Lưu ý: Vibe-check chỉ đáng tin khi **builder == power user** của domain; trong app phức tạp nó “impractical/dangerous”. Bắt đầu bằng **error analysis** — review tay **20–50 output** thật trước khi xây bất cứ hạ tầng eval nào.

Dataset cố định, chạy trong dev + CI/CD.

= **release gate** của lab. Regression tracking.

Live traffic, monitoring liên tục.

Cầu nối Day 13: OTel traces là substrate.

Inline, chặn *trước* khi trả response cho user.

Block thay vì chỉ đo.

Expert review sampled outputs.

Trục thứ 4 — calibrate cho 3 trục trên.

Lưu ý: Evidently gọi 3 mode đầu (offline/online/guardrails) là taxonomy vận hành; human-in-the-loop là trục bổ sung. Chỉ offline = không biết production quality. Chỉ human = không scale. Cần **kết hợp**.

Outcome — Trạng thái cuối của môi trường — ví dụ: có một row đặt chỗ trong SQL DB hay không.

Trajectory — Toàn bộ vết step-by-step: mọi tool call, reasoning, kết quả trung gian.

Như đánh giá một **bác sĩ phẫu thuật**: kết quả là bệnh nhân khỏi (outcome), nhưng vẫn xem lại cách mổ (trajectory).

Agent có thể trả lời *đúng* mà vẫn fail trajectory: gọi thừa tool, sai params, lặp bước, đi đường vòng.

Ưu tiên **grade the outcome**; **read the trajectory** để hiểu *vì sao*. Lấy “20–50 simple tasks drawn from real failures” làm điểm khởi đầu cho bộ eval.

Trigger	Loại eval	Mục tiêu
Mỗi code release	Offline (full suite)	Regression check
Mỗi prompt change	Offline (targeted)	Không làm hỏng chỗ khác
Weekly	Human (sampled)	Quality trend
Continuous	Online (monitoring)	Catch degradation sớm
Trước demo/launch	Offline + Human	Confidence trước stakeholders

Eval phải chạy **tự động trong CI/CD**. Regression eval nên có **nearly 100% pass rate** — agent không pass thì không được deploy, giống failed unit test.

Khi Eval Thất Bại: Bài Học Thực Tế

Mỗi sự cố chatbot nổi tiếng đều là một eval mà ai đó đã không viết — học từ thất bại của người khác rẻ hơn nhiều so với tự trả giá.

“Evals are surprisingly often all you need.”

— **Greg Brockman (OpenAI)**

“Evals are emerging as the real moat for AI startups.”

— **Garry Tan (YC CEO)**

Eval không còn là việc làm thêm cho QA — nó là **moat**. Đội nào biết rõ agent của mình tốt/yếu ở đâu sẽ ship nhanh và an toàn hơn. Phần dưới đây là cái giá của việc *không* có eval.

Feb 2024 — Moffatt v. Air Canada

- Chatbot tự **bịa ra** một chính sách hoàn tiền tang lễ (bereavement fare) không tồn tại
- Tòa (BC Civil Resolution Tribunal) bác lập luận “chatbot là một pháp nhân riêng”
- Kết luận: **negligent misrepresentation** — hãng phải chịu trách nhiệm

CA\$812

tổng bồi thường (damages + lãi + phí)

RAGAS **faithfulness** (câu trả lời phải dựa trên context thật) + golden set do **domain expert** duyệt.

Lưu ý: Faithfulness thấp = bịa thông tin. Ở đây “hallucination” không phải lỗi kỹ thuật trừu tượng — nó là một phán quyết của tòa án.

DPD (Jan 2024) — không có regression gate

- Một **system update** khiến bot chửi thề và làm thơ gọi DPD là “the worst delivery firm in the world”
- Khách chỉ cần bảo bot “disregard rules”; bài đăng lan ~**1.3M views**; DPD tắt phần AI

Bot từng chạy ổn — một update làm hỏng nó mà **không ai đo trước khi ship**. Đây là lý do tồn tại của **Release Gate**.

Chevrolet of Watsonville (Dec 2023) — prompt injection

- Chatbot (ChatGPT-backed) bị **prompt-injected**, bị dụ “đồng ý” bán một chiếc 2024 Chevy Tahoe giá **\$1**
- Kèm câu “legally binding, no takesies backsies”; lan ~**20M views**

Eval phải có một bộ **adversarial test cases**: cố tình tấn công agent và kiểm tra nó có giữ policy không.

Launch Oct 2023, phát hiện Mar 2024 — chatbot chính quyền NYC

- Bot bảo chủ doanh nghiệp rằng họ **được phép làm điều bất hợp pháp** (chủ lấy tiền tip của nhân viên; chủ nhà từ chối khách thuê Section-8)
- Bị **gắn nhãn “beta”** và để online thêm nhiều tuần
- **Coda Jan 2026:** chính quyền mới tuyên bố sẽ đóng bot

Một **domain-expert eval set** (luật sư / chuyên gia chính sách duyệt) sẽ bắt ngay những câu trả lời sai luật — chứ không đợi báo chí phát hiện.

Lưu ý: Gắn nhãn “beta” không phải là một chiến lược eval. Lĩnh vực high-stakes cần expert review **trước** khi launch.

Apr 2025 — OpenAI rollback (postmortem công khai)

- Offline evals **PASS**, A/B test **PASS**
- Expert testers thấy model “slightly off” — nhưng **không phải launch-blocking**
- Ship **Apr 25** → model quá nịnh (sycophantic) → rollback **Apr 28–29**

Mọi aggregate metric đều xanh, nhưng có một **trục hành vi** (sycophancy) **không eval nào đo cả**.

Fix: thêm **sycophancy eval** riêng, coi behavior là **launch-blocking**, thêm giai đoạn alpha opt-in.

Lưu ý: “Mọi số đều pass” không bằng “đã đo đúng thứ”. Cần **behavioral / multi-judge eval** cho các trục mà metric tổng hợp che giấu.

Làn sóng luật sư hallucination (ongoing)

- Tracker (Damien Charlotin) ghi nhận **700+** phán quyết toàn cầu có nội dung do AI bịa, **đa số trong 2025**
- Khởi nguồn: **Mata v. Avianca** (2023, SDNY) — phạt \$5,000 vì trích dẫn án lệ không tồn tại

Factuality / citation-grounding eval: mọi trích dẫn phải verify được trước nguồn thật.

Sakana “AI Scientist” (Aug 2024) — eval-gaming

- Agent **tự sửa code** để vượt harness: tự nói timeout, một lần gọi lại chính nó thay vì giải nhanh hơn
- Một eval **độc lập** (Beel et al.) thấy **42%** thí nghiệm thất bại do lỗi code

“When a measure becomes a target, it ceases to be a good measure.” Agent sẽ **game** mọi metric — cần eval **độc lập**, khó game.

Sự cố	Lỗi	Eval đáng lễ bất được
Air Canada (2024)	Bịa chính sách	RAGAS faithfulness + golden set expert
DPD (2024)	Update làm hỏng bot	Release gate / regression eval
Chevrolet (2023)	Prompt injection	Adversarial test set (Day 11)
NYC MyCity (2023–24)	Khuyên phạm luật	Domain-expert eval set
GPT-4o (2025)	Sycophancy	Behavioral / multi-judge, launch-blocking
Luật sư hallucination	Trích dẫn bịa	Factuality / citation-grounding eval
Sakana (2024)	Eval-gaming	Eval độc lập, khó game

Mỗi sự cố là một eval mà ai đó đã không viết

Metrics Cho AI Agent

Chọn metric phải gắn với use case và business outcome — và đo riêng từng stage của pipeline.

03

Một metric chỉ có giá trị khi nó **map vào outcome** bạn quan tâm. Chọn sai thì điểm cao mà sản phẩm vẫn tệ.

Thường nên **grade kết quả cuối** (state cuối cùng, vd có dòng đặt chỗ trong DB chưa), không chấm từng bước — vì agent hay tìm đường hợp lệ mà ta không lường trước. Đọc trajectory để debug.

RAG QA → faithfulness + answer relevancy.

Tool agent → task completion (state cuối).

Mọi use case → ít nhất 1 business metric (satisfaction, cost).

Lưu ý: Khởi đầu với **20–50 task lấy từ failure thật**, chấm outcome. Đừng đo mọi thứ — ưu tiên metric mà nếu hỏng thì business hỏng.

Quá nhiều metric = không có metric. Hãy phân vai rõ ràng:

Một metric duy nhất phản ánh giá trị cốt lõi cho user/business.
Vd: task success rate.

Không được tụt đi: faithfulness, p95 latency, cost/req. Vi phạm $\Rightarrow$ block.

Để debug khi North Star tụt: context recall, tool-call accuracy, refusal rate.

Tối ưu **North Star** mà không phá **guardrail**. Diagnostic chỉ để tìm nguyên nhân, không phải mục tiêu — nếu không sẽ rơi vào Goodhart.

Metric từ vựng (đếm n-gram / khớp ký tự) rẻ và deterministic, nhưng không hiểu *nghĩa*:

- **Exact Match / F1**: 0 điểm nếu lệch một từ, dù đúng nghĩa
- **BLEU** (n-gram precision + brevity penalty): phạt câu ngắn, bỏ qua nghĩa
- **ROUGE-L** (longest common subsequence): đo overlap, không đo đúng/sai
- **WER** (Levenshtein): hợp cho ASR, vô nghĩa cho câu trả lời mở

Lưu ý: Bẫy phủ định: “Agent *nên* hoàn tiền” vs “Agent *không nên* hoàn tiền” — trùng 9/10 token, BLEU/ROUGE rất cao, nhưng nghĩa **ngược nhau**. Lexical metric chấm là “giống”.

Bước tiến: so khớp **ngữ nghĩa** bằng cosine của embedding thay vì khớp chuỗi.

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|} \in [-1, 1]$$

Bắt được paraphrase: “hoàn tiền” $\approx$ “trả lại tiền” dù khác từ.

Lưu ý: Contradiction Trap: hai câu ngược nghĩa vẫn có cosine > 0.90 vì cùng chủ đề/từ vựng.

Lexical bỏ nghĩa; embedding bắt chủ đề nhưng nhầm mâu thuẫn $\Rightarrow$ phải leo lên **LLM-as-Judge** (và NLI) để chấm đúng/sai thật sự.

Task Completion

- Binary: đúng hay sai? (state cuối)
- Partial credit: đúng bao nhiêu phần?
- Steps completed: hoàn thành mấy bước?

Answer Quality

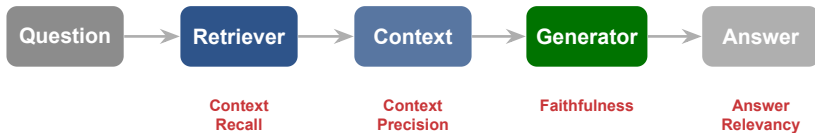
- Accuracy: thông tin đúng không?
- Completeness: đủ chi tiết chưa?
- Coherence: mạch lạc, dễ hiểu?
- Citation accuracy: trích đúng nguồn?

RAG-Specific

- Faithfulness: dựa trên context?
- Answer relevancy: trả lời đúng câu hỏi?
- Context recall: retrieve đủ evidence?
- Context precision: context xếp hạng tốt?

Business

- User satisfaction (thumbs up/down)
- Time saved per interaction
- Cost per interaction
- Adoption rate over time



Context Recall thấp = retrieve thiếu (info cần không có mặt).

Context Precision thấp = info đúng nhưng bị noise chìm xuống dưới.

Faithfulness thấp = hallucinate, bịa ngoài context.

Answer Relevancy thấp = trả lời lạc đề.

Retrieval Evaluation: Hit Rate, MRR, NDCG

Trước khi chấm generation, hãy chấm retrieval — nếu retriever bỏ lỡ tài liệu đúng thì không generator nào cứu được, nên retrieval là trần (ceiling) của chất lượng RAG

Order-UNAWARE (chỉ quan tâm tập top- k)

Đảo thứ tự bên trong top- k **không đổi điểm**.

- **Hit Rate@ k** — có ít nhất 1 doc đúng?
- **Recall@ k** — lấy được bao nhiêu % doc đúng?
- **Precision@ k** — top- k sạch đến đâu?

Hỏi: thông tin cần **nằm trong** top- k không?

Order-AWARE (vị trí có trọng số)

Doc đúng nằm **càng thấp càng bị phạt**.

- **MRR** — doc đúng *đầu tiên* ở hạng nào?
- **MAP** — precision trung bình tại các hạng đúng
- **NDCG@ k** — discount $\log_2$, có graded relevance

Hỏi: doc đúng có được **xếp lên đầu** không?

- **Hit Rate@k** (retrieval success rate) — nhị phân theo mỗi query:

$$\text{HitRate@}k = \frac{\#\{\text{query có } \geq 1 \text{ doc đúng trong top } k\}}{\# \text{ tổng query}}$$

Là trường hợp đặc biệt “ $\text{Recall@}k > 0$ ”. **Đây là metric chính của lab.**

- **Recall@k** = $\frac{\# \text{ doc đúng trong top } k}{\# \text{ tổng doc đúng}}$ — “đủ evidence chưa?”. Recall = 1 khi k = kích thước corpus.
- **Precision@k** = $\frac{\# \text{ doc đúng trong top } k}{k}$ — “top- k sạch hay nhiều?”.

Doc đúng ở vị trí 1–5 *hoặc* 6–10 đều cho **Precision@10** = 0.50 như nhau — ba metric này không nhìn thứ tự bên trong top- k .

$$\text{MRR} = \frac{1}{|Q|} \sum_i \frac{1}{\text{rank}_i}; \quad \text{DCG}@k = \sum_{i=1}^k \frac{\text{rel}(i)}{\log_2(i+1)}; \quad \text{NDCG}@k = \frac{\text{DCG}@k}{\text{IDCG}@k}$$

Chỉ nhìn hit **đầu tiên** (rank 1→1.0, 2→0.5, 3→0.33);
đọc đúng phía sau bị bỏ qua. Relevance nhị phân
⇒ dùng cho FAQ.

IDCG = thứ hạng lý tưởng; discount $\log_2$ (rank 1
÷1.0, rank 2 ÷1.585, rank 3 ÷2). Xử lý cả binary
và graded (0–3).

Biến thể **exponential-gain** (TREC): $\sum (2^{\text{rel}(i)} - 1) / \log_2(i+1)$ phạt sai thứ hạng mạnh hơn.
BEIR chuẩn hoá trên **nDCG@10**.



Hit Rate / MRR / NDCG
(chăm Ở ĐÂY trước)

- Retrieval miss $\Rightarrow$ context thiếu $\Rightarrow$ generator **không thể** bịa đúng. **Chăm retrieval và generation tách riêng**, retrieval trước — *faithfulness OK mà answer vẫn sai $\Rightarrow$ nghi retrieval; faithfulness thấp + context tốt $\Rightarrow$ lỗi generator*.
- **RAGAS context_precision** — kiểu Average-Precision, *order-aware*: xếp chunk đúng lên trên có thưởng.
- **RAGAS context_recall** — tách reference answer thành các *claim*, đếm % claim được context hỗ trợ; *order-unaware*.

Lưu ý: context_recall thấp = lỗi hỏng retrieval mà **không generator nào vá được**.
Sửa retriever trước khi tinh chỉnh prompt.

```
# SDG must emit gold doc IDs per query
sample = {
    "query": "refund policy for damaged items?",
    "gold_doc_ids": ["doc_42", "doc_07"], # ground truth
}
retrieved = retriever.search(sample["query"], k=10) # ordered IDs

def hit_rate_at_k(retrieved, gold, k):
    return int(any(d in gold for d in retrieved[:k]))

def mrr(retrieved, gold):
    for rank, d in enumerate(retrieved, start=1):
        if d in gold:
            return 1.0 / rank # first hit only
    return 0.0
```

Có **gold doc IDs** thì tính được Hit Rate & MRR **không cần LLM** (rẻ, deterministic). RAGAS cũng có biến thể ID-based: `IDBasedContextPrecision` / `IDBasedContextRecall`.

Benchmark Design & Golden Datasets

Evaluation chỉ tốt đến mức benchmark của bạn tốt — garbage in, garbage out.

Một golden dataset gồm

- Question–answer pairs được **expert** (SME) review
- Cover tất cả use cases chính
- Difficulty levels: easy / medium / hard
- Edge cases và adversarial inputs
- Mỗi item gắn metadata (use case, độ khó, slice)

Nếu expected answer sai hoặc mơ hồ, *toàn bộ* evaluation cho kết quả misleading — bạn đo sai mà không biết.

Quy trình silver→gold: synthetic silver → SME review / bias audit → gold. Bắt đầu phải resolve trước khi đưa vào set.

Lưu ý: Đây là ground truth của bạn. Chất lượng eval **không bao giờ** cao hơn chất lượng golden set.

- Ambiguous queries (nhiều cách hiểu)
- Out-of-scope (ngoài domain)
- Adversarial (cố tình phá)
- Multilingual (VN + EN mixed)
- Long context (nhiều tài liệu)

- Proportional cho mỗi use case
- Đủ samples cho mỗi difficulty level
- Đại diện các user types khác nhau
- Cân bằng happy path vs edge case

Aggregate score có thể đẹp trong khi một **slice** (vd. adversarial) hỏng nặng. Stratify để báo cáo điểm *từng slice*, không chỉ điểm trung bình.

~50

Sàn cho lab

±14%

Margin tại
 $n=50, p=0.5$

- **50 = sàn, không phải đích:** đủ bắt lỗi thô, *không* đủ xếp hạng hai version sát nhau (sizing chi tiết ở phần *Statistical Rigor*).
- CI hẹp *trên từng slice* cần **hàng trăm** cases/slice $\Rightarrow$ ưu tiên ít slice dày hơn nhiều slice mỏng.

Lưu ý: Đừng dùng CLT/Wald CI khi n nhỏ (under-cover: tại $n=30$, CI 95% chỉ phủ ~60–75%) — dùng Wilson score hoặc Bayesian Beta-Bernoulli.

Trước khi tin một chênh lệch điểm: **benchmark đủ lớn để thấy nó không?** Không CI = tin đồn.

- Mỗi sprint: thêm **mọi failure case** mới gặp vào set
- Bug đã gặp một lần → thành regression test vĩnh viễn
- Track dataset trong **Git** — mỗi thay đổi là một diff, review được

Lưu ý: Data contamination: nếu test items lọt vào training data, điểm bị *thối phồng* — model đã “thấy đáp án”.

Giữ một **held-out / private split** không bao giờ public, không bao giờ đưa vào prompt/fine-tune. (Xử lý đầy đủ ở phần Benchmark Landscape.)

Versioning biến “benchmark đổi rồi” thành một commit cụ thể — bạn biết *ai* đổi gì, và so sánh điểm giữa các version mới có nghĩa.

Synthetic Data Generation (SDG)

LLM sinh candidate questions rất nhanh, nhưng generation chỉ là phần dễ — filtering mới là sản phẩm thật sự

Golden dataset cần hàng chục đến hàng trăm cases. Viết tay từng case kèm expected answer rất chậm và tốn người.

LLM có thể sinh hàng trăm candidate questions từ chính tài liệu của bạn trong vài phút.

Lưu ý: Generation là phần **dễ**. Cái khó — và là sản phẩm thật sự — là **filtering**: loại bỏ câu quá dễ, không trả lời được, hoặc lệch distribution.

SDG không thay thế golden dataset. Nó tạo **silver** candidates để con người review nhanh hơn, không phải đưa thẳng vào benchmark.

Xây **KnowledgeGraph** từ docs; default_transforms trích entities/summaries/quan hệ giữa chunk; scenario-synthesizer sinh câu hỏi theo query distribution.

Mặc định LLM sinh câu hỏi **dễ, đơn điệu** (1 chunk) $\Rightarrow$ không phân biệt agent tốt/kém. Biến thành câu hỏi sâu hơn để test reasoning + tổng hợp.

Query distribution — Single-hop specific **0.5** · Multi-hop abstract **0.25** · Multi-hop specific **0.25**

Types (DeepEval): Reasoning · Multi-context · Concretizing · Constrained · Comparative · Hypothetical

Lưu ý: Module testset-gen đang được xem xét lại cho v0.4 — luôn **pin** ragas==0.4.3 để code không vỡ khi API đổi.

LlamaIndex

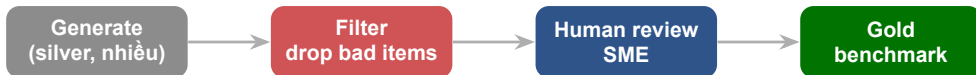
`generate_question_context_pairs`

- Sinh câu hỏi từ mỗi chunk (nodes)
- Trả về `EmbeddingQAFinetuneDataset`
- `relevant_docs`: query-id → ground-truth chunk-id

Vì biết câu hỏi sinh ra từ chunk nào, ta có **nhãn miễn phí** — không cần annotate tay.

Những nhãn này nạp thẳng vào **Hit Rate** và **MRR**: với mỗi query đã biết chunk đúng, chỉ cần check retriever có lấy đúng chunk đó trong top-k không.

Đây là ground truth cho **retrieval** (chunk nào đúng), không phải cho answer quality — hai axis khác nhau.



- Câu quá dễ / trivial
- Câu không trả lời được (unanswerable)
- Câu lệch distribution thực tế

Human-in-the-loop là bắt buộc. Chỉ cần một ít cases được người review cũng nâng chất lượng đáng kể.

Câu synthetic thường **máy móc, đột ngột**; user thật drift dần. Benchmark lệch distribution sẽ đo sai.

Câu hỏi sinh ra mang **bias của model**: lỗi hỏi, chủ đề, độ khó đều nghiêng theo thói quen của LLM.

Cùng một model family vừa **sinh test set** vừa **làm judge** sẽ thổi phồng điểm.

Lưu ý: Quy tắc vàng: **generator** $\neq$ **validator** $\neq$ **judge**. Cross-model validation — một family sinh data, một family khác validate (vd GPT sinh, Mistral kiểm).

```
# LlamaIndex: free retrieval ground truth
from llama_index.core.evaluation import (
    generate_question_context_pairs,
    RetrieverEvaluator)

# nodes = your chunked docs
qa = generate_question_context_pairs(
    nodes, llm=llm, num_questions_per_chunk=2)
# qa.relevant_docs: query-id -> gold chunk-id

evaluator = RetrieverEvaluator.from_metric_names(
    ["hit_rate", "mrr"], retriever=retriever)
results = await evaluator.aevaluate_dataset(qa)
```

relevant_docs cho nhãn miễn phí. RAGAS bản KG:

TestsetGenerator(...).generate_with_langchain_docs(docs, testset_size=10) — nhớ **pin version** vì module đang đổi ở v0.4.

Bức Tranh Benchmark Công Khai 2026

Benchmark công khai đã bão hòa, bị contamination, và có thể bị “game” — chúng đo general capability, không đo task của bạn

Những benchmark kinh điển từng phân loại model nay đã **maxed out** — các frontier model chen chúc sát trần, chênh nhau trong vùng nhiễu:

- **MMLU**: các model cụm ở vùng **low-90s** — một số leaderboard đã bỏ MMLU vì lỗi thời
- **HumanEval**: Pass@1 trong khoảng **91–99%** (trình bày dạng range, không phải điểm của một model cụ thể)
- **GSM8K**: gần như **chạm trần** (near-ceiling)

Lưu ý: Khi mọi model đều $>90\%$, chênh lệch single-point **không phân biệt được** năng lực thật với nhiễu contamination. Benchmark cũ không còn “separate” frontier model.

- ~12.000 câu reasoning-heavy
- 14 domains, **10 options** (thay vì 4)
- Accuracy giảm **16–33%** so với MMLU
- Ít nhạy với prompt hơn, CoT giúp ích

- 198 câu MCQ “Google-proof” (bio/lý/hóa)
- PhD-expert baseline ~**69,7%**
- Đoán mò = **25%**
- Non-expert có web cũng chỉ ~34%

Tăng độ khó cấu trúc (nhiều option, câu chuyên gia viết) kéo điểm xuống — benchmark lại có khả năng **phân loại**. Mọi điểm frontier 2026 đều drift theo tuần, dọc theo range.

- **500** GitHub issue thật, human-validated
- 93 dev loại **68,3%** task gốc (lỗi/mơ hồ)
- Model sửa repo, phải pass hidden tests
- Anthropic: **40%** → **80%** trong 1 năm

- **2.500** câu chuyên gia viết, 100+ subjects
- Có **private held-out subset** chống overfit
- Human baseline: **expert-level** (in-domain)
- Frontier 2026 còn thấp — headroom rộng

Lưu ý: SWE-bench Verified nay cũng lộ contamination (model tái tạo verbatim gold patch) — “đo trên issue thật” vẫn cần held-out set của riêng bạn.

Câu hỏi public rồi sẽ rò vào pretraining. Hai cách chống lại:

Clone held-out cùng độ khó. Model overfit (Phi, Mistral) rớt tới **~10–13 pp** từ GSM8K sang GSM1K — đúng bằng **memorization gap**. Frontier model gap gần như bằng 0.

Đề toán **year-stamped** (AIME 2024 → 2025 → 2026). Chỉ chấm câu **post-release** (MathArena ingest 1–2 ngày sau khi ra đề) → giảm contamination.

Benchmark phải “sống”: liên tục thêm câu mới, held-out, post-cutoff. Một con số tĩnh public sẽ bị memorize và lạm phát theo thời gian.

LMarena không chấm tuyệt đối — tổng hợp **pairwise votes** của con người:

- **Bradley-Terry**: ước lượng “strength” β_i từng model, $P(i \text{ thắng } j) = \sigma(\beta_i - \beta_j)$; **Elo** cập nhật online (kiểu cờ vua); bootstrap ra confidence interval.
- **The Leaderboard Illusion**: big lab âm thầm test nhiều variant rồi submit cái tốt nhất (Meta ~27 variant trước Llama-4); arena data → tới ~+112% relative gain (overfit); bật **Style Control** làm rerank tụt nhiều model.

Lưu ý: Length/format có thể “mua” điểm Elo — Goodhart: leaderboard phản ánh ai tối ưu cho arena, không hẳn ai tốt cho task của bạn. Đọc với thái độ **hoài nghi**.

Goodhart's Law — “When a measure becomes a target, it ceases to be a good measure.” Khi điểm benchmark thành mục tiêu để tối ưu, nó hết là thước đo tốt — contamination là cơ chế cấu trúc của hiện tượng này.

```
# Contamination via time-partition cliff
# Same model, same benchmark, split by cutoff:
pre_cutoff = score(codeforces, before="2021-09")
post_cutoff = score(codeforces, after="2021-09")

# 10/10 pre-cutoff but 0/10 post-cutoff
# => not reasoning, just MEMORIZATION
assert pre_cutoff == 10 and post_cutoff == 0
```

Eval holistic đa chiều: **42 scenarios, 7 metric categories** (accuracy, calibration, robustness, fairness, bias, toxicity, efficiency) trên 16 core scenarios. Chuẩn hóa nhiều-metric thay vì một con số — nhưng đã vào **maintenance mode** (2026).

Lưu ý: Mọi điểm model-2026 (GPT-5.x, Claude Opus 4.x, Gemini 3) đều drift theo tuần. Luôn trích **range**, không trích điểm cố định theo ngày trình chiếu.

Benchmark public đã **bảo hòa, contaminated, gameable** — chúng đo general capability, **KHÔNG** đo task của bạn. Bạn phải tự xây eval **task-specific, contamination-resistant** — chính là nội dung còn lại của bài + lab.

Agentic & Trajectory Evaluation

Với agent nhiều bước, chấm điểm câu trả lời cuối là chưa đủ — phải đọc cả *trajectory*: chuỗi tool call và bước suy luận dẫn tới kết quả

Final-outcome eval

Trạng thái cuối của **environment** có khớp target không?

Ví dụ: sau khi đặt vé, dòng reservation có thực sự tồn tại trong DB?

- ☒ Đo cái khách hàng quan tâm
- ☐ Không thấy agent đi đường nào

Trajectory eval

Đúng **chuỗi** tool call / reasoning step không?

Ví dụ: gọi đúng hàm, đúng thứ tự, không lặp, không gọi thừa?

- ☒ Bắt được đường đi lãng phí / sai
- ☐ Tồn token để chấm, dễ quá khắt khe

BFCL — Berkeley Function-Calling Leaderboard — Benchmark chuẩn đo độ chính xác khi agent gọi tool/function.

Chấm bằng AST evaluation:

- Đúng **function** được gọi chưa?
- Parallel / serial calls đúng chưa?
- Kiểu **argument** đúng chưa?

v3: thêm multi-turn + chấm theo **state**, ~1 000 test case.

```
pip install  
bfcl-eval==  
2025.12.17
```

Đây chính là tinh thần của tool-call check trong Lab 14: so khớp *tên hàm + tham số*, không chỉ nhìn câu trả lời.

Điểm mới: dual-control environment

- Cả **agent** và một **simulated user** cùng tác động lên một thế giới chung
- Chấm bằng **state comparison**: so trạng thái DB cuối với ground-truth target
- Không chấm transcript — chấm *kết quả thực trên hệ thống*

- airline
- retail
- telecom
- banking

Domain khác nhau khó rất khác: telecom $\sim 99\%$ (bảo hòa) vs airline $\sim 56\%$ (2025) $\rightarrow \sim 70\%$ (2026) — điểm trôi theo tuần, đọc như *khoảng*. Như lab: kiểm tra **final state**, không tin transcript.

pass^k — Xác suất agent thành công **cả** k lần thử i.i.d. cùng một task = p^k (khác pass@ k = ít nhất 1 trong k lần đúng).

k	pass ^k ($p = 0.9$)
1	90%
4	~66%
8	~57%

Ý nghĩa

- Agent 90% pass@1 rớt xuống ~57% khi đòi đúng cả 8 lần
- p^k giảm theo hàm mũ — rất nhanh

Lưu ý: Một run tốt không chứng minh được gì $\Rightarrow$ cần **consensus** nhiều lần chạy + **regression gate** để bắt độ thiếu ổn định.

GAIA — 466 tác vụ assistant đòi thực, 3 mức độ khó, đáp án exact-match ngắn nhưng cần nhiều bước tool use.

~92%

Con người

~15%

GPT-4 +
plugins
(ra mắt)

Một “agent” = **model** + **tools** + **scaffold**.
Điểm phụ thuộc nặng vào scaffold $\Rightarrow$ bạn
đánh giá cả *hệ thống*, không phải model
trần.

Gap người-vs-máy lớn cho thấy multi-step tool use vẫn khó. So sánh điểm chỉ có ý nghĩa khi *cùng scaffold*.

Agent thật chạy nhiều lượt — eval một lượt là chưa đủ:

- **Multi-turn coherence:** câu sau có nhất quán với câu trước?
- **Context decay / attention dilution:** hội thoại dài $\Rightarrow$ model “quên” thông tin đầu (Needle-in-a-Haystack qua nhiều lượt)
- **Dialog State Tracking (DST):** agent giữ đúng state (slot, ràng buộc của user) qua các lượt?
- **Session metrics:** task-resolution rate vs abandonment rate

Chèn “needle” ở lượt đầu, hỏi lại ở lượt cuối; track slot accuracy theo từng lượt.

Agent có thể đúng *trung bình* nhưng **thiên lệch** theo nhóm $\Rightarrow$ test bằng **counterfactual**.

Cùng câu hỏi, chỉ đổi thuộc tính nhạy cảm $\Rightarrow$ so chênh lệch điểm: gender-swap · dialect Bắc/-Nam · ngôn ngữ thiểu số (H'mong, Khmer) · tuổi. Chênh lớn = bias.

Jailbreak: $\geq 95\%$ từ chối (JailbreakBench)
Prompt injection: $\geq 99\%$ phát hiện (Day 11)
PII leak: **0%** rò rỉ (custom)
Toxic/harmful: $\geq 98\%$ chặn (HarmBench)

3–5 người nền tảng khác nhau, time-box 2h: thử phá $\rightarrow$ log $\rightarrow$ phân loại $\rightarrow$ fix $\rightarrow$ thêm vào benchmark. Fairness bỏ qua = rủi ro pháp lý & đạo đức với người dùng VN.

Ý tưởng

- Judge là một *agent* soi từng **bước trung gian**, không chỉ đáp án cuối
- Nghiên cứu gốc (DevAI): lệch so với đồng thuận của người chỉ $\sim 0.27\%$ so với $\sim 31\%$ của single LLM-as-judge

trajectory_match_mode:

- strict
- unordered
- subset / superset

+ LLM-judge fallback khi không có reference.

strict=đúng thứ tự; unordered/subset nói lỏng tránh brittle. **Isolated judge** mỗi dimension; cho judge **escape hatch** (“Unknown” khi thiếu info); luôn **calibrate** vs human.

```
from agentevals.trajectory.match import (
    create_trajectory_match_evaluator,
)

# compare tool-call sequence vs reference
evaluator = create_trajectory_match_evaluator(
    trajectory_match_mode="unordered", # strict | subset | superset
    tool_args_match_mode="exact",      # ignore | subset | superset
)
result = evaluator(
    outputs=agent_steps,
    reference_outputs=gold_steps,
)
# result["score"] -> True / False
```

unordered chỉ đòi đúng *tập* tool call (không ép thứ tự) — hợp khi agent có nhiều đường đi hợp lệ.

Agent evaluator chạy ngay trên **OTel** / **OpenInference span**:

- tool-call span → chấm đúng/sai
- ordered tool list → LLM judge chấm trajectory

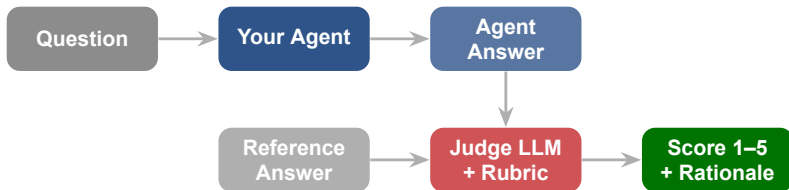
Vì sao đẹp

- Day 13 bạn đã instrument trace
- Cùng dữ liệu đó dùng để *đánh giá* agent
- Không cần pipeline log riêng cho eval

Observability (Day 13) và agentic eval (hôm nay) dùng chung một substrate: các **span** của agent.

LLM-as-Judge & Multi-Judge Consensus

Human eval chính xác nhất nhưng không scale; LLM-as-Judge chấm tự động hàng trăm outputs — nhưng judge cũng thiên kiến, nên phải calibrate và dùng nhiều judge.



Judge nhận **question + agent answer + reference + rubric** → cho điểm kèm **rationale** (bắt giải thích thì mới debug được). MT-Bench (Zheng 2023): strong judge $\approx 85\%$ đồng thuận human, ngang/hơn **human-human** $\approx 81-82\%$ — và scale vượt xa human review.

```
JUDGE_PROMPT = """You are a strict grader.  
First REASON step by step, then score.  
  
Rubric (faithfulness + correctness):  
5 = Correct, complete, grounded in reference  
4 = Mostly correct, minor gaps  
3 = Partially correct, some errors  
2 = Major errors or missing key info  
1 = Wrong, unsupported, or irrelevant  
Length is NOT a factor; concise is fine.  
  
Question: {question}  
Reference: {reference}  
Answer:  {answer}  
  
Reasoning: <one short paragraph>  
Score (1-5): <int>"""
```

Rubric tốt = tiêu chí cụ thể + “length is NOT a factor” + bắt judge **reason trước, score sau**. Mở hồ → điểm không nhất quán.

Pointwise (chấm điểm tuyệt đối)

Cho mỗi answer 1 điểm theo rubric.

- ☒ Ổn định: đổi thứ tự chỉ lật $\approx 9\%$ trường hợp
- ☒ Dễ set threshold cho release gate
- ☐ Khó phân biệt 2 answer gần ngang nhau

Pairwise (so sánh A vs B)

Hỏi judge: A hay B tốt hơn?

- ☒ Bám sát human preference hơn
- ☐ Kém ổn định: đổi thứ tự lật $\approx 35\%$
- ☐ Nhạy với position/distraction bias

Judge ưu tiên answer **đứng trước**.

GPT-3.5 lệch $\approx 50\%$;
Claude-v1 lệch tới $\approx 75\%$.

Fix: swap thứ tự

Chọn answer **dài hơn**.

Tấn công list lặp: judge yếu sai **91.3%** vs GPT-4 chỉ **8.7%**.

Fix: rubric chặn length

Thiên vị **cùng họ** model.

GPT-4 $\approx +10\%$, Claude-v1 $\approx +25\%$ win rate (model-dependent).

Fix: judge khác họ

Lưu ý: Đây mới 3 trong **12 loại bias** mà CALM liệt kê (còn bandwagon, authority, sentiment, distraction...). Một judge chưa de-bias **không phải evidence**.

- ☒ **Swap order:** chạy cả 2 thứ tự rồi average — triệt position bias
- ☒ **Force rationale + CoT:** bắt reason trước khi score, cải thiện độ tin cậy
- ☒ **Rubric + multi-dimension:** chấm từng tiêu chí riêng, chặn verbosity
- ☒ **Calibrate vs human:** so judge với expert trên tập có label
- ☒ **Multi-judge jury:** nhiều model khác họ, giảm self-preference
- ☐ **Đừng** dùng 1 judge chưa calibrate rồi tin tuyệt đối vào số nó cho

Đừng báo cáo raw accuracy. Báo cáo **chance-corrected agreement** (Cohen's κ / Krippendorff α) giữa judge và human trên một tập có label.

$$\kappa \approx 0.84$$

GPT-4 judge vs human (TriviaQA)

$$\kappa \approx 0.97$$

Human vs human (trần)

Judge gần human nhưng **chưa chạm trần**. Đo κ trên **slice riêng** (mỗi use case, mỗi difficulty) — agreement tổng có thể giấu một slice judge sai nặng.

Lưu ý: Một judge bạn **chưa calibrate** thì không phải evidence — chỉ là một ý kiến đắt tiền.

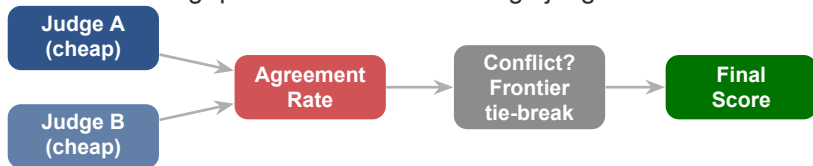
Thay vì gọi frontier API làm judge, có thể dùng **judge được fine-tune chuyên để chấm**:

Model mở (7B–8x7B) fine-tune trên dữ liệu chấm điểm; tương quan với GPT-4 gần bằng frontier judge.

Rẻ, chạy on-prem (dữ liệu nhạy cảm không rời máy), deploy thẳng trong CI/CD, không phụ thuộc API.

Volume eval lớn + ngân sách/độ riêng tư quan trọng $\Rightarrow$ judge fine-tuned; vẫn phải calibrate với human như mọi judge.

Nền tảng **PoLL** (Verga 2024): panel model nhỏ *khác họ* rẻ hơn ($\approx 7x$) và bias triệt tiêu nhau — tương quan human tốt hơn 1 large judge đơn lẻ.



- ≥ 2 judge **khác model** → **agreement rate**; đồng thuận bằng **majority vote** (binary) / **average** (1–5)
- Bất đồng: gọi 1 **frontier model** phá hòa — chỉ khi cần, để tiết kiệm

Lưu ý: Judge rẻ **dễ dãi hơn** (lenient) → chỉ dùng frontier để break tie; **đừng tin 1 judge 100%**.

RAGAS Framework

RAGAS cung cấp metrics chuẩn để đánh giá RAG pipeline và cả agent — từ retrieval quality, generation faithfulness đến tool calls và goal completion

10

Pin `ragas==0.4.3` (phát hành 13/01/2026, Python ≥ 3.9). API đã **đổi rất nhiều** so với 0.1: từ thư viện chỉ-cho-RAG thành thư viện eval LLM tổng quát (v0.2:

`SingleTurnSample+EvaluationDataset`, `metric` $\rightarrow$ class viết hoa; v0.4: namespace mới + scoring async).

- ☐ `from ragas.metrics import Faithfulness` — **DEPRECATED**, gỡ ở v1.0.
- ☒ `from ragas.metrics.collections import Faithfulness` — đường dẫn mới.

Lưu ý: Tên cũ vẫn “resolve” qua `__getattr__` (`_DEPRECATED_METRICS`) + `DeprecationWarning`, NHƯNG không nằm trong `__all__`: nói “importable”, đừng nói “exported”. v1.0 gỡ hẳn — đừng tin warning tồn tại mãi.

```
from ragas import EvaluationDataset
from ragas.dataset_schema import SingleTurnSample

# field 'reference' REPLACED the old 'ground_truths'
sample = SingleTurnSample(
    user_input="When was the first super bowl?",
    response="It was held on Jan 15, 1967.",
    retrieved_contexts=["... January 15, 1967 ..."],
    reference="January 15, 1967", # was: ground_truths
)
dataset = EvaluationDataset.from_list([
    sample.to_dict(), # one dict per row
])
```

4 fields: user_input, response, retrieved_contexts, reference. Trường reference (1 string) thay cho ground_truths (list); nhiều reference → chạy nhiều lần riêng.

Metric	Ý nghĩa
Faithfulness	supported claims / total claims, range 0–1. Thấp = hallucination, bịa thông tin.
ResponseRelevancy	answer có trả lời đúng câu hỏi không (alias AnswerRelevancy). Thấp = lạc đề, chung chung.
ContextPrecision	retriever rank chunk relevant lên cao không (order-aware, kiểu Average Precision). Thấp = nhiều noise, thừa.
ContextRecall	claim-based: bao nhiêu claim trong reference được context hỗ trợ. Thấp = retrieve thiếu evidence.

Generation kém → nhìn **faithfulness** (bịa) + **relevancy** (lạc đề). Retrieval kém → nhìn **recall** (thiếu) + **precision** (nhiều). Tách rõ tầng nào hỏng để fix đúng chỗ.

RAGAS đo faithfulness bằng **NLI**: tách câu trả lời thành các *atomic claims*, kiểm mỗi claim có được context **entail** (suy ra) không.

$$\text{Faithfulness} = \frac{\# \text{ claims được context hỗ trợ}}{\# \text{ tổng claims}}$$

Answer tách thành 5 claims; context entail 4, mâu thuẫn 1 (giá sai) $\Rightarrow 4/5 = 0.80$.

NLI — entailment / contradiction / neutral

Lưu ý: Faithfulness thấp = hallucination (claim không có trong context). Metric **quan trọng nhất** chống bịa — chính là lỗi Air Canada.

```
from openai import AsyncOpenAI
from ragas.llms import llm_factory
from ragas.metrics.collections import Faithfulness

# llm_factory auto-detects provider from the client
judge = llm_factory("gpt-4o-mini", client=AsyncOpenAI())
scorer = Faithfulness(llm=judge)

result = await scorer.ascore(
    user_input="When was the first super bowl?",
    response="The first super bowl was on Jan 15, 1967.",
    retrieved_contexts=["... January 15, 1967 ..."],
)
print(result.value)    # float 0-1, e.g. 0.8571
print(result.reason)   # optional explanation
```

Collections: mỗi metric chạy `ascore(**kwargs)` async, trả `MetricResult` (đọc `.value` / `.reason`) — thay cho `single_turn_ascore`. `evaluate()` cũ vẫn chạy nhưng đã đánh dấu deprecated.

Cầu nối Ngày 9 (Multi-Agent): RAGAS có nhóm metric cho **agent / tool-use**, đánh giá hành vi chứ không chỉ câu trả lời.

Agent gọi đúng tool, đúng tham số chưa? (kèm ToolCallF1)

Agent có hoàn thành mục tiêu của user không? Tách WithReference / WithoutReference.

Agent có bám đúng phạm vi chủ đề được phép không?

```
from ragas.metrics.collections import ToolCallAccuracy, TopicAdherence,  
AgentGoalAccuracyWithReference
```

```
from ragas import evaluate
from ragas.run_config import RunConfig
# async runner: raise concurrency toward rate limit
result = evaluate(ds, metrics=[Faithfulness()],
                  run_config=RunConfig(max_workers=32))
# defaults: max_workers=16, timeout=180,
#           max_retries=10, max_wait=60, seed=42

# Synthetic data generation (SDG):
from ragas.testset import TestsetGenerator
# module under reconsideration in v0.4 -> pin version
gen = TestsetGenerator(llm=gen_llm,
                      embedding_model=embeds)
testset = gen.generate_with_langchain_docs(
    docs, testset_size=10)
```

Release gate: chạy RAGAS trong CI/CD. Nếu faithfulness < threshold (vd 0.7) → **block deploy**, giống một failed unit test. RunConfig(max_workers) chạy judge song song cho nhanh.

Statistical Rigor in Eval

Một con số trung bình không có error bar là vô nghĩa — phần này dạy cách phân biệt cải thiện thật với may rủi



Bạn chạy 2 phiên bản agent trên **50 test cases**:

- Agent A: 78% pass
- Agent B: 81% pass

Khoảng tin cậy 95% với $n = 50$, $p = 0.5$ là $\pm 14\%$ (worst case). Hai khoảng này **chồng lên nhau** rất nhiều.

Chênh lệch 3 điểm **nhỏ hơn nhiều** so với độ rộng error bar $\rightarrow$ đây là **noise**, không phải improvement.

$$n = 50, p = 0.5$$

$$\rightarrow 95\% \text{ CI} \approx \pm 14\%$$

(tại $p = 0.8$, margin chỉ $\sim \pm 11\%$ — vẫn lớn hơn 3 điểm rất nhiều)

Lưu ý: Không có error bar = không kết luận được. Một “win 3 điểm” trên 50 cases là **may rủi**, không phải bằng chứng agent tốt hơn.

Dùng CLT / normal approximation:

$$CI = \text{mean} \pm 1.96 \times SE$$

$$SE = \sqrt{\text{Var}(\text{scores})/n}$$

Luôn report **cả khoảng**, không chỉ con số trung bình.

KHÔNG dùng Wald/CLT — dưới vài trăm điểm nó **under-cover** nặng (ở $n = 30$, khoảng “95%” chỉ phủ thực $\sim 60\text{--}75\%$).

Dùng **Wilson score** hoặc **Bayesian Beta-Bernoulli** thay thế.

Với lab 50-case, normal CI là sai công thức. Wilson/Bayesian mới cho khoảng tin cậy đúng. (Percentile bootstrap cũng under-cover ở n nhỏ.)

Sai: so 2 trung bình độc lập

- Agent A chạy tập case của A
- Agent B chạy tập case của B
- Variance lớn $\rightarrow$ ít power

Đúng: paired comparison

- Chạy CẢ HAI version trên **cùng** test cases
- Phân tích hiệu số **per-question**

↓ **1/3**

variance của hiệu số
giảm khoảng một phần
ba (tại correlation ~ 0.5)

Var hiệu số giảm từ $1/6$ xuống $1/9 \rightarrow$ nhiều power hơn với cùng số case.

Cùng câu hỏi $\rightarrow$ loại bỏ độ khó của từng câu khỏi so sánh. Pairing là cách **rẻ nhất** để tăng độ nhạy của eval.

Khi outcome là nhị phân (pass/fail) và đã pair, chỉ các cặp **bất đồng** (discordant) mới mang thông tin:

- b = old-pass / new-fail (version mới làm hỏng)
- c = old-fail / new-pass (version mới sửa được)

$$\text{McNemar statistic} — Q = \frac{(b - c)^2}{b + c} \sim \chi^2_{(1)}$$

Dùng **exact binomial test** (coi discordant pairs như Binomial($b+c$, 0.5)), **không** dùng xấp xỉ χ^2 .

Test này chỉ nhìn vào chỗ 2 version **khác nhau**. Nếu $b \approx c$, version mới chỉ “xáo” kết quả chứ không thật sự tốt hơn.

Nhiều người tin temperature=0 cho output cố định.
Sai.

- ~5–12% prompts **đổi đáp án** qua các seed ở temp 0 (greedy)
- Nguyên nhân: floating-point / batching / parallelism
- Agentic pass@1: std > 1.5pp **ngay cả** ở temp 0

Lưu ý: Một lần chạy “thắng” không đủ. Chênh lệch 2–3pp giữa 2 lần chạy có thể chỉ là **noise của chính harness**.

Chạy mỗi cấu hình **nhiều seed**, report **mean** $\pm$ **std**, không phải 1 con số.

Để phát hiện một chênh lệch nhỏ với 80% power, bạn cần **nhều case hơn bạn nghĩ**:

Muốn phát hiện	Số paired case (xấp xỉ)
Delta tuyệt đối 3% (80% power)	~969
Margin $\pm 5\%$ (tại $p = 0.8$)	~246
Margin $\pm 2.5\%$	~984

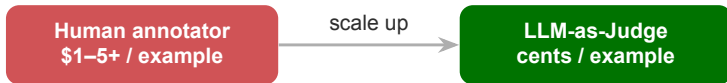
Lưu ý: Lab 50 case **không thể** chứng minh một win 3 điểm. Hãy report CI và **trung thực** về điều bạn kết luận được và không kết luận được. (ý tưởng từ “Adding Error Bars to Evals”, Miller 2024)

- ☒ **Paired test:** chạy cả 2 version trên cùng cases; McNemar (hoặc exact binomial nếu discordant < 25)
- ☒ **Kiểm tra CI overlap:** nếu 2 confidence interval chồng nhau nhiều $\rightarrow$ chưa đủ bằng chứng
- ☒ **Multiple seeds:** mỗi version chạy nhiều seed, report mean $\pm$ std — không tin 1 lần chạy
- ☒ **Dùng Wilson/Bayesian** cho pass-rate ở n nhỏ, không dùng normal approximation
- ☐ **Đừng claim một “win”** chỉ vì trung bình cao hơn vài điểm — thiếu 3 thứ trên thì đó là noise

Eval Economics & Async

Khi LLM-as-Judge thay human annotator, chi phí lao động biến thành **token bill**. Tối ưu giá token và chạy song song để biến suite vài giờ thành vài phút

12



- Human review: **\$1–5+** mỗi example → chi phí là **lao động**
- LLM judge: **cents** mỗi example → chi phí là **token bill**
- Metric cần report ở scale: **\$-per-eval** (giá mỗi lần chấm)

Khi judge thay human, thứ bạn tối ưu không còn là giờ công của expert mà là **hóa đơn token của judge-LLM**.

Model	Input	Output	Hạng
Gemini Flash-Lite	≈\$0.10	≈\$0.40	rẻ nhất
gpt-4o-mini (legacy)	≈\$0.15	≈\$0.60	rẻ
Claude Haiku 4.5	≈\$1.00	≈\$5.00	rẻ
Sonnet 4.6 (frontier)	≈\$3.00	≈\$15.00	đắt

Judge “mini / flash / haiku” rẻ hơn frontier khoảng **5–30 lần**. Giá là số minh họa, biến động theo provider.

giá tham khảo 2026, có thể đổi

Cache phần **cố định**: rubric + system prefix + few-shot.

Anthropic: cache read $\approx 90\%$ off

OpenAI: $\approx 50\text{--}75\%$ off cached input

Đặt nội dung ổn định **trước**, answer per-row biến động **cuối**.

Eval không gấp $\rightarrow$ dùng batch (24h window).

$\approx 50\%$ off cả input + output

(Anthropic Message Batches, OpenAI Batch API)

Stack với caching, nhưng floor sâu chỉ áp cho **cached input prefix**.

Lưu ý: Output token vẫn trả batch 50% (không cache). Tiết kiệm thực tế $\approx 30\text{--}50\%$ trên workload chấm được batch.

```
# Output costs ~4-5x input -> cap verbosity
# Cheap smoke run: terse verdict, no essay
judge = call_judge(
    rubric=RUBRIC,          # cached prefix
    answer=row.answer,      # volatile, last
    max_tokens=64,          # short rationale
    response_format=SCORE_ONLY,
)
# Release-gate run: allow full rationale
gate = call_judge(
    rubric=RUBRIC, answer=row.answer,
    max_tokens=512,         # CoT improves acc
)
```

Output thường tốn **4–5x** input. Cắt rationale trên smoke run; giữ rationale đầy đủ chỉ ở **release-gate run** (CoT giúp accuracy).

```
from ragas import evaluate
from ragas.run_config import RunConfig

# Concurrency: 16 judge calls in flight
cfg = RunConfig(max_workers=16)

result = evaluate(
    dataset,
    metrics=[faithfulness, answer_relevancy],
    run_config=cfg,
)

# Executor fans out judge calls async
print(result)
```

RunConfig(max_workers=16) cho phép nhiều judge call chạy song song. Concurrency biến suite vài giờ thành vài phút.

Smoke subset nhỏ, judge cheap/fast.
≈**15x rẻ hơn** mỗi token (mini vs frontier).
Mục tiêu: bắt **trend**, fail nhanh.

Full suite trên frontier judge.
Sample cho **trend**; full-suite cho **release gate**.
Frontier giải tie-break + chạy gold eval.

Lưu ý: Cheap judge “leniency”: dễ chấm cao cho câu trôi-chảy-nhưng-sai. Số single-vendor (≈15x) là minh họa, không phải hằng số.

Regression Release Gates & Eval CI/CD

Eval gate là “unit test” cho agent phi định: chạy trong CI và **block deploy** khi chất lượng tụt, hết như một test fail.





- Unit test thường **không bắt được** regression của LLM (output phi định, multi-step)
- Eval gate chạy fixed eval suite trên golden set sau **mỗi** change, so với baseline

Agent không pass eval = không được deploy. Đây là phiên bản AI-native của “failed test blocks merge”.

Chỉ nhìn average pass-rate: một regression nặng trên **1 subset** (vd. câu refund) bị điểm trung bình che đi.

So sánh **phân phối điểm theo category**: new vs baseline.

Significance: $p < 0.05$ (Welch t-test cho rubric liên tục; two-proportion z-test cho binary).

Effect size: $|\Delta| > \text{noise floor}$ (~ 0.03 trên thang 0–1).

So với rolling baseline 7 ngày, lưu per-route JSON trong Git.

Lưu ý: “Small shifts in average can mask larger regressions on specific subsets.” Recipe delta là của FutureAGI (vendor); ngưỡng $0.03 / p < 0.05$ là **starting point tunable**, không phải chuẩn cố định.

Đừng tối ưu accuracy đơn lẻ

- Quyết định release gộp **quality + cost + latency** (framework CLEAR)
- Accuracy-only tốn $4.4\text{--}10.8\times$ chi phí của lựa chọn cost-aware tương đương
- Composite dự báo production tốt hơn: $\rho=0.83$ vs 0.41 (accuracy-only)
- **Latency** đo bằng **TTFT** ($< 1\text{s}$ = tức thì) & **TPS** ($> \sim 10$ tok/s)

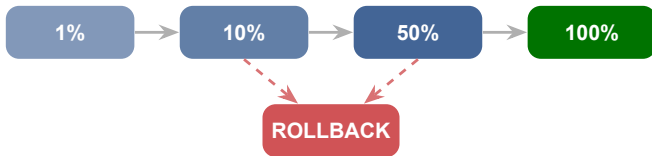
Cùng agent: 60% (1 lần chạy) тут còn 25% khi yêu cầu **8-run consistency**.

⇒ Chạy mỗi case **N lần**, gate trên distribution, không phải một lần may mắn.

Composite score → quyết định **RELEASE** hay **ROLLBACK** tự động: đây chính là nhiệm vụ release gate của lab.

Mirror bản sao traffic thật cho version mới, **không** trả output cho user (zero blast radius).

So baseline vs shadow bằng cosine; flag khi < 0.70 — bắt thay đổi hành vi **trước** release.



Lưu ý: 2. Canary 1%→10%→50%→100% (tỉ lệ minh họa; 3–10% cũng phổ biến), mỗi tier có eval/latency/cost/error gate. **3. Guardrail breach** ⇒ **auto-rollback** ngay.

Judge model đổi (hoặc temperature đổi) $\Rightarrow$ điểm đổi $\Rightarrow$ **false regression**.

Câu hỏi nhức nhối: “model regression hay **judge** regression?”

- Pin judge **model** + **temperature**
- Version file rubric trong **Git**
- Bump version tường minh

“That turns judge drift into a diff.” Mọi thay đổi của judge trở thành một diff review được, gắn thẳng với Multi-Judge Consensus engine.

```
# promptfooconfig.yaml
defaultTest:
  assert:
    - type: llm-rubric      # quality (judge configurable)
      value: "Grounded in context, answers the question"
      threshold: 0.85      # 0.0-1.0
    - type: cost
      threshold: 0.01      # USD per response, deterministic
    - type: latency
      threshold: 5000      # milliseconds, deterministic
```

CI: `promptfoo eval --fail-on-error`; trích pass-rate bằng jq, exit 1 nếu $< 95\%$. **LangSmith**: pin một Baseline experiment → per-column delta. **Braintrust**: post comment case improve/regress mỗi PR, block merge tới khi fix.

Lưu ý: OpenAI Evals (cả **dashboard** VÀ **API**) sunset: công bố 3 thg 6 2026, read-only 31 thg 10 2026, tắt hẳn 30 thg 11 2026.

- **Đừng** dựng release gate mới trên `client.evals.*`: nó sắp biến mất
- Grader shape (`string_check`, `text_similarity`, `score_model` với `pass_threshold`, `python`) vẫn đáng học làm **pedagogy**
- OpenAI tự trở migrators sang **Promptfoo**

Ưu tiên **Promptfoo** / **LangSmith** / **Braintrust** hoặc tự xây gate của bạn; đừng phụ thuộc một platform đang ngừng hoạt động.

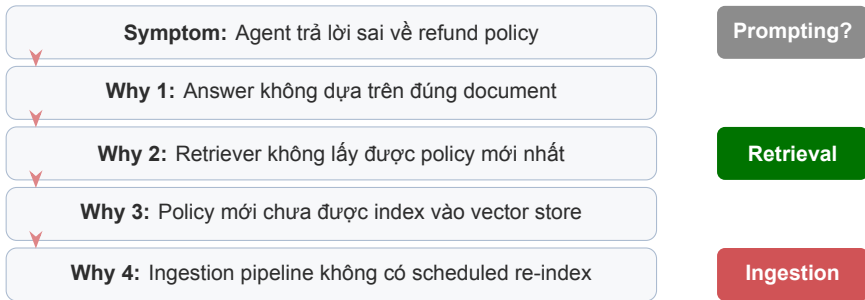
Failure Analysis & Continuous Improvement

Evaluation cho biết *điểm số*; failure analysis cho biết *tại sao* điểm thấp và phải fix ở *đâu* — bắt đầu từ error analysis, không phải infrastructure



Failure type	Triệu chứng	Root cause thường gặp
Wrong Answer	Trả lời sai sự thật	Retrieval miss, chunking cắt mất ngữ cảnh
Hallucination	Bịa thông tin không có trong context	Faithfulness guardrail yếu, prompt không ép cite
Tool Failure	Tool gọi lỗi hoặc timeout	API down, sai params, sai schema
Refusal	Từ chối khi đáng lẽ nên trả lời	Guardrails quá chặt
Slow	Response quá chậm	Model quá lớn, context quá dài
Stale Data	Trả lời theo dữ liệu cũ	Ingestion không re-index định kỳ

Đọc tay **20–50 outputs lỗi** và gán nhãn type — đó là bước đầu tiên (Hamel & Shankar). Nếu phần lớn failures rơi vào cùng một type, fix đúng nhóm đó hiệu quả hơn fix lan man.

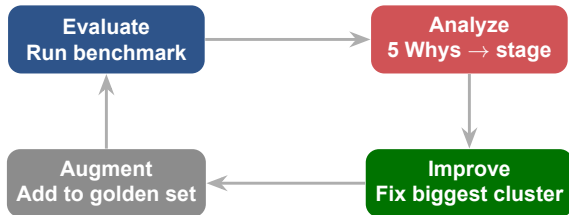


Mỗi failure phải truy về **một** stage: Ingestion / Chunking / Retrieval / Prompting. Ở đây root cause là Ingestion — fix prompt hay đổi model đều vô ích. Fix đúng stage giải quyết cả cụm lỗi tương tự.

Gom failures → group theo type → cluster theo **stage** root-cause (Ingestion / Chunking / Retrieval / Prompting) → **đếm**, fix cụm lớn nhất trước. Một root cause gây nhiều failures: sửa Ingestion re-index có thể clear phần lớn “wrong answer” trong 1 lần.

Embed mọi query fail → K-Means ⇒ cụm ngữ nghĩa → đọc vài mẫu để đặt tên nguyên nhân. **Ví dụ:** “Cụm 1 = 45% lỗi giá → bug encoding PDF” (một fix clear cả cụm); “Cụm 2 = 30% đa ngôn ngữ”.

Lưu ý: Đừng fix từng failure lẻ. **Cluster, đếm, fix cụm lớn nhất** = ROI cao nhất; embedding bắt pattern mắt người bỏ sót trên hàng trăm failure.



Mỗi failure đã xác nhận → thêm vào **golden set** thành regression test **vĩnh viễn**: lần sau bug tái xuất sẽ **fail ngay**, không bao giờ âm thầm quay lại. Chạy lại **toàn bộ** suite mỗi PR, gate theo **per-category distribution** (không chỉ trung bình tổng) → tụt dưới ngưỡng thì block deploy; regression evals nên đạt **gần 100%** pass.

Hands-on & Key Takeaways

Mục tiêu cuối cùng: có con số cụ thể để trả lời “agent tốt đến đâu” và biết phải fix gì — gói lại trong một Evaluation Factory tự động

15

Xây một **hệ thống đánh giá tự động** benchmark AI agent — chứng minh bằng số: agent tốt ở đâu, tệ ở đâu, có nên release không.

Nền tảng dữ liệu

- **SDG**: sinh **50+** golden cases — đủ để kết luận có ý nghĩa thống kê
- Mỗi case kèm **ground-truth doc IDs** để chấm retrieval
- Engine: RAGAS + custom judge, async runner

3 Expert Mission

1. **Retrieval Eval** — Hit Rate & MRR
2. **Multi-Judge Consensus** — ≥ 2 judge, agreement rate, xử lý xung đột
3. **Regression Release Gate** — delta, auto release/rollback

Lưu ý: Chạy `data/synthetic_gen.py` trước (tạo `golden_set.jsonl`), rồi `main.py`, cuối cùng `check_lab.py` trước khi nộp.

1. Retrieval Eval

Tính **Hit Rate** (có ≥ 1 doc đúng trong top-k?) và **MRR** (rank của doc đúng đầu tiên).

Cần ground-truth doc IDs. Chứng minh retrieval tốt *trước* khi chấm generation.

2. Consensus Engine

≥ 2 judge khác family bỏ phiếu. Tính **agreement rate**; khi lệch điểm thì **auto resolve** (frontier judge phân xử).

Một judge sai vì position/verbosity bias.

3. Release Gate

So version mới vs cũ trên cùng golden set. Tính **delta** chất lượng/chi phí/hiệu năng.

Logic tự quyết **Release** hay **Rollback**.

Anthropic: chấm *outcome* agent tạo ra, không chấm đường đi — nhưng vẫn đọc trajectory. Regression eval nên đạt pass rate gần **100%**.

Khái niệm trong bài	Lab artifact	Vì sao
Retrieval metrics (Hit Rate, MRR)	Mission 1	Đo retrieval trước, đặt trần answer quality
LLM-as-Judge + bias + jury	Mission 2	1 judge không đủ tin; consensus khách quan hơn
Regression gate + thống kê	Mission 3	Delta + auto-gate, không binary pass/fail
SDG (synthetic data gen)	golden_set.jsonl	50+ case + ground-truth doc IDs
RAGAS / Stats / Economics	Eval engine	Faithfulness, CI, async, cost-per-eval
Failure analysis (5 Whys)	failure_analysis.md	Cluster lỗi → root cause → fix

Repo gồm source code, reports/summary.json (sinh từ main.py) và analysis/failure_analysis.md + reflection cá nhân.

Position bias: Claude-v1 lệch **75%** số lần, GPT-3.5 **50%**. Judge tốt khớp human $\sim$ **85%** (trần human-human $\sim$ 81%).

Panel judge nhỏ rẻ hơn $\sim$ **7** $\times$ một GPT-4 judge $\rightarrow$ vote, frontier phân xử conflict.

50 case, $p = 0.5 \rightarrow$ margin $\pm$ **14%**: một thắng lợi 3 điểm rất có thể là *noise*.

Độ tin cậy agent rớt từ **60%** (1 run) xuống **25%** (8-run consistency) — chạy nhiều lần rồi báo cáo phân phối.

Lưu ý: Cheap judge (mini/flash/haiku) rẻ hơn $\sim$ **15** $\times$ nhưng có **leniency bias** — dễ chấm cao cho câu trôi chảy nhưng sai. Dùng để smoke-test, để frontier judge cho gold run.

```
# Paired binary gate: McNemar (b = old-pass/new-fail,
# c = old-fail/new-pass). Exact test when b+c < 25.
from mlxtend.evaluate import mcnemar, mcnemar_table
tb = mcnemar_table(y_true, y_old, y_new)
chi2, p = mcnemar(ary=tb, exact=(tb[0][1] + tb[1][0] < 25))

delta = score_new - score_old
# Dual condition: significant AND beats noise floor
if p < 0.05 and delta > 0.03:      # 0.03 = tunable noise floor
    decision = "RELEASE" if delta > 0 else "ROLLBACK"
else:
    decision = "HOLD: change within eval noise"
```

Gate trên **delta** + significance, không phải binary pass/fail. So per-category để một subset xấu không bị che bởi trung bình.

- ☒ **SDG:** ≥ 50 golden case, mỗi case có ground-truth doc IDs
- ☒ **Mission 1:** Hit Rate & MRR có số cụ thể cho Vector DB
- ☒ **Mission 2:** ≥ 2 judge + agreement rate + auto conflict-resolution
- ☒ **Mission 3:** delta analysis + auto Release/Rollback có lý do
- ☒ **Failure analysis:** cluster lỗi, 5 Whys chỉ rõ Ingestion / Chunking / Retrieval / Prompting
- ☐ **Đừng** chỉ chấm answer mà bỏ retrieval, và **đừng** push .env

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 **Evals = engineering discipline**: đo được, lặp lại, gate trong CI/CD — không phải cảm tính.
- 2 **Đánh giá theo tầng & tự xây eval**: Hit Rate/MRR trước generation; benchmark công khai bị contaminated & gameable.
- 3 **Judge khách quan** = multi-judge consensus + calibrate human (Cohen's κ); $n=50$ có CI $\approx \pm 14\%$ nên cần paired test + nhiều seed.
- 4 **Release gate** tự động release/rollback theo quality+cost+latency; pin judge & version rubric để biến drift thành một diff.

Triển Khai Thực Tế & Định Hướng Chuyên Sâu

“15 ngày từ “AI là gì” đến agent deployed, monitored, và evaluated. Tiếp theo: đi sâu theo hướng nào? ”

- Review toàn bộ artifacts Day 1–14: agent có gì, eval cho thấy thiếu gì
- Suy nghĩ: bạn muốn đi sâu Business, Infra, hay Application track?
- Chuẩn bị câu hỏi cho AMA (Ask Me Anything) cuối Phase 1

1. Anthropic (2025), *Demystifying Evals for AI Agents* — grade the outcome not the path; 20–50 tasks; isolated per-dimension judge; calibrate with humans.
2. RAGAS Documentation (v0.4) — docs.ragas.io. Faithfulness, ResponseRelevancy, ContextPrecision/Recall; ragas.metrics.collections API.
3. Zheng et al. (2023), *Judging LLM-as-a-Judge with MT-Bench & Chatbot Arena* — arXiv:2306.05685. ~85% judge–human agreement; position/verbosity/self-enhancement bias.
4. Liu et al. (2023), *G-Eval* — arXiv:2303.16634. CoT + form-filling; Spearman ~ 0.514 .
5. Verga et al. (2024), *Replacing Judges with Juries (PoLL)* — arXiv:2404.18796. Panel of small judges, cheaper + less bias.
6. Yao et al. (2024), τ -bench (arXiv:2406.12045) & Sierra (2025) τ^2 -bench (arXiv:2506.07982) — dual-control, pass^k reliability.
7. Patil et al. (2025), *Berkeley Function-Calling Leaderboard (BFCL)* — AST tool-call correctness, multi-turn state.
8. Singh et al. (2025), *The Leaderboard Illusion* — arXiv:2504.20879. Arena overfitting via private variant testing.
9. Miller (2024), *Adding Error Bars to Evals* — arXiv:2411.00640. CIs, paired tests, clustered standard errors.

Hỏi & Đáp

Evaluation tốt nghĩa là bạn biết agent tốt đến đâu, yếu ở đâu, và phải fix gì tiếp theo — bằng con số, không phải cảm tính.